



获取模块元数据

北京理工大学计算机学院
金旭亮

概述



Java 9 引入了用于表示运行时模块的新类型 `java.lang.Module`。它的实例使开发者能够做到以下几点：



动态查询和获取模块的名称、导出或开放的包和使用或提供的服务信息；

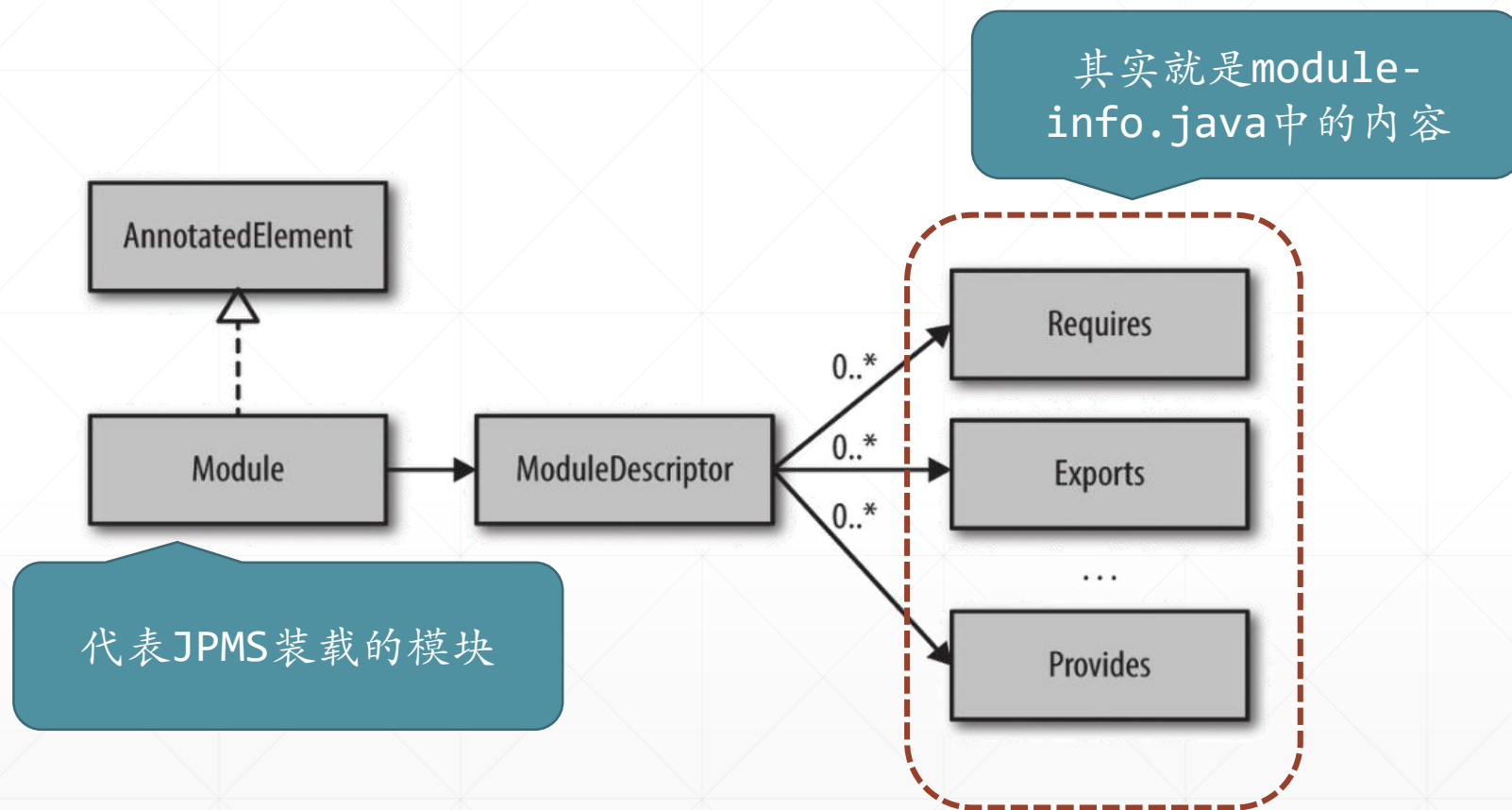


访问模块包含的资源；



动态导出和开放包、动态增加可读性和服务。

Module及其相关类的简化类图



获取Module实例

获取Module实例的一种方法是在任意一个 Class 实例上调用getModule()方法，即可得到该类所属的模块：

```
Module module = String.class.getModule();
```

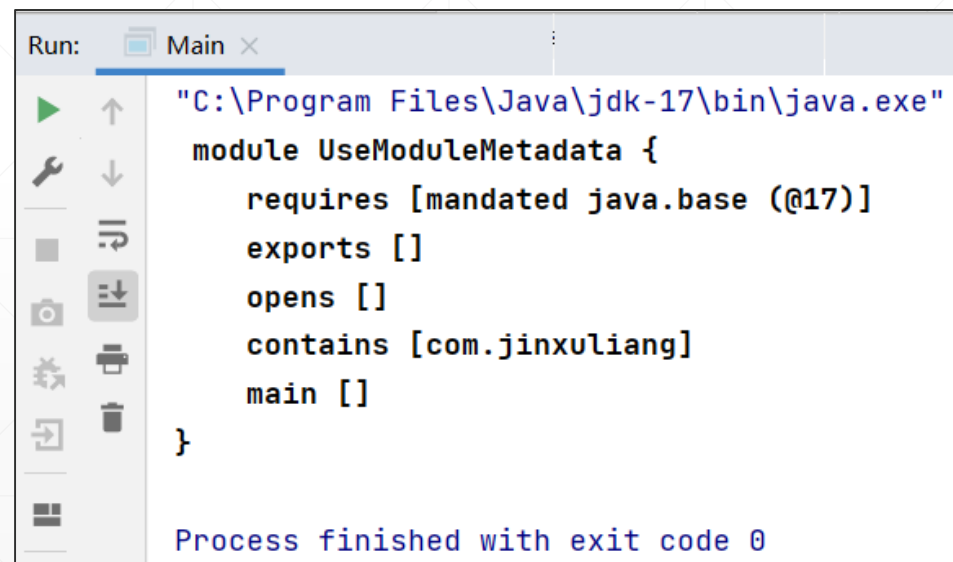
有了Module的实例，就可以调用它来查询模块的元数据：

```
private static void showModuleInfo(){  
    Module module = Main.class.getModule();  
    //获取模块的元数据  
    System.out.println(describe(module));  
}
```

查询模块的元数据示例

```
public static String describe(Module module) {  
    ModuleDescriptor md = module.getDescriptor();  
    if (md == null)  
        return "UNNAMED module { }";  
    return " module " + md.name()  
        + " {\n"  
        + "\trequires " + md.requires() + "\n"  
        + "\texports " + md.exports() + "\n"  
        + "\topens " + md.opens() + "\n"  
        + "\tcontains " + md.packages() + "\n"  
        + "\tmain " + toString(md.mainClass()) + "\n"  
        + "}";  
}
```

运行截图：

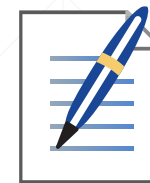


The screenshot shows a 'Run' window with a tab labeled 'Main'. The output text is as follows:

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
module UseModuleMetadata {  
    requires [mandated java.base (@17)]  
    exports []  
    opens []  
    contains [com.jinxuliang]  
    main []  
}
```

At the bottom of the window, it states: "Process finished with exit code 0".

在运行时动态修改模块描述符



除了分析模块的属性，还可以使用 `Module` 中的以下方法来修改这些属性。



`addExports`，向指定模块导出一个包。



`addOpens`，向指定模块开放一个包。



`addReads`，使模块可以读取另一个模块。



`addUses`，让模块使用一个服务。

动态添加导出示例代码

```
Module target = ...; //以某种方式获得想要导出到的模块  
Module module = getClass().getModule(); //获取当前类的模块  
module.addExports("包名", target);
```

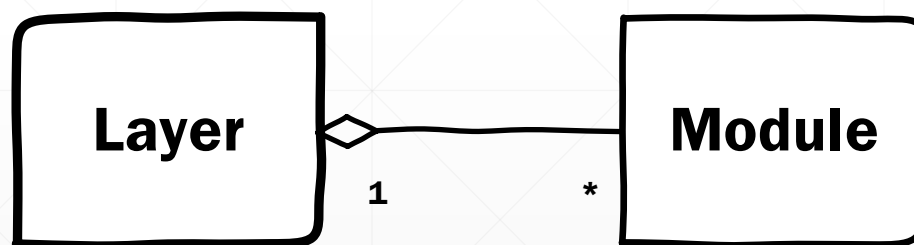


当尝试将导出添加到正在执行调用的当前模块之外的任何其他模块时，JVM就会引发异常。

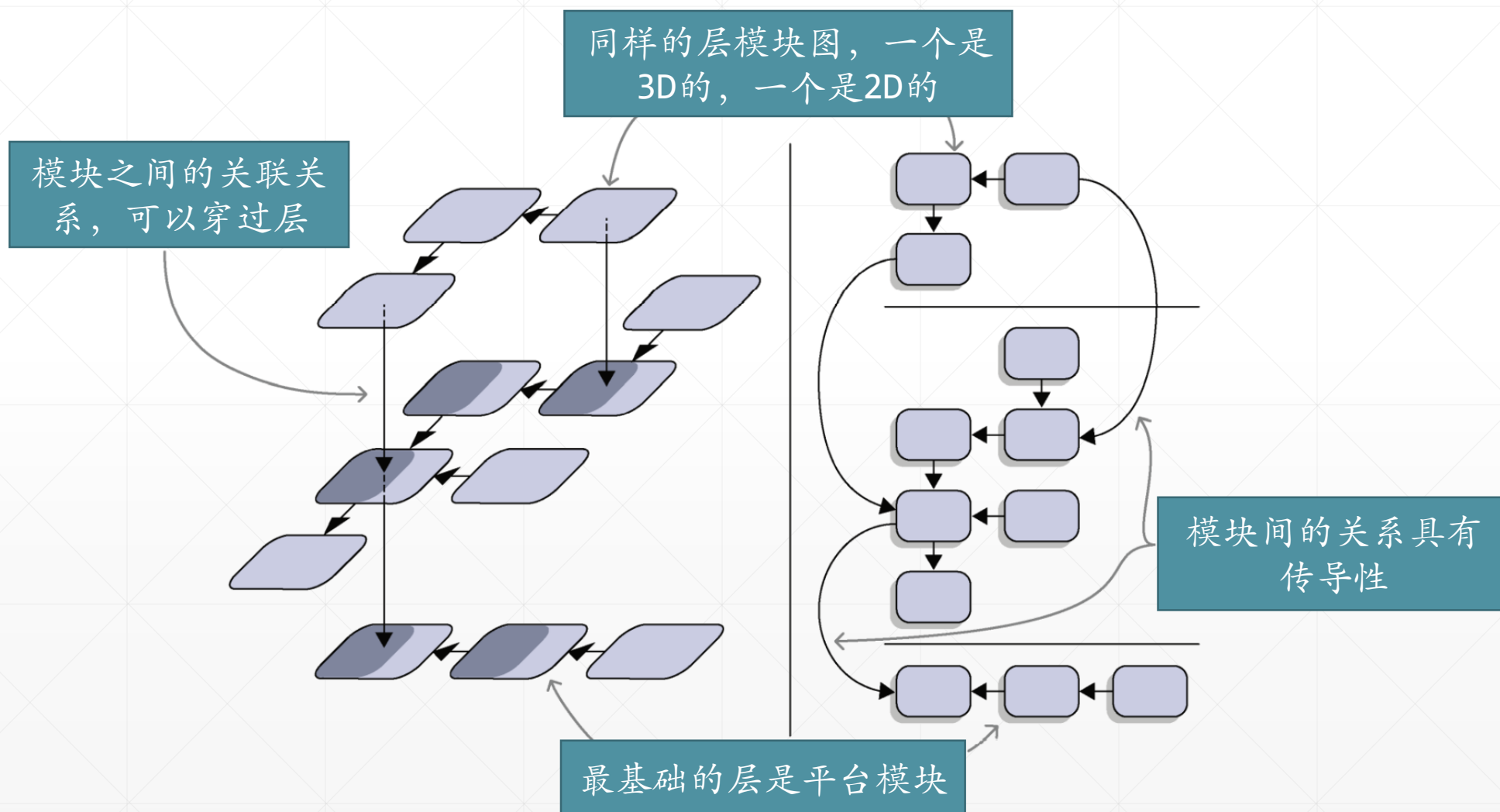
层



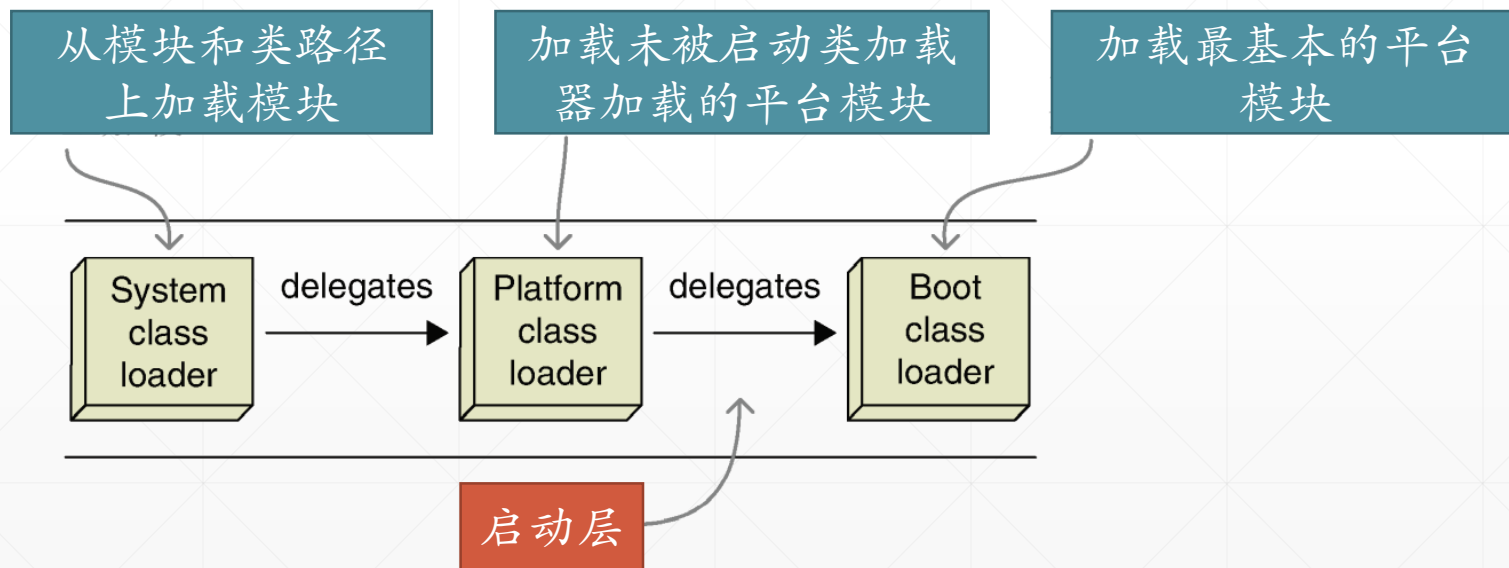
层是模块的集合，JPMS使用它来加载和统一管理模块。



层包容有特定的加载器，可以动态加载和移除模块，因此，可以基于它实现支持动态插拔的插件化系统。



启动时，JVM创建一个初始层，即**启动层**，包含根据命令行选项解析的应用程序模块和平台模块。



查询层

`java.lang.ModuleLayer` 实例代表层，利用它可以查询层所包含的以下信息：



模块列表

- `modules()` 方法以 `Set` 集合的形式返回该层包含的模块。
- `findModule(String)` 方法在该层以及它的所有祖先层中查找具有特定名称的模块。



`parent()` 方法以 `List<ModuleLayer>` 的形式返回该层的父模块层



每个模块的类加载器可以通过用模块名调用 `findLoader(String)` 的方式确定。



`configuration()` 方法返回一个 `Configuration` 实例，获取配置信息。

获取层的实例

```
Class<?> type = // ..... 任意类型

//获取指定类型所在的模块，进而获取模块所归属的层
ModuleLayer layer = type.getModule().getLayer();

//..... 利用层提供的API干活
```



如果希望访问启动层，则可以调用静态方法
`ModuleLayer::boot`

基于层查询模块

//获取层中所包容的所有模块

```
private static String describe(ModuleLayer layer) {  
    return layer  
        .modules().stream() Stream<Module>  
        .map(Module::toString) Stream<String>  
        .collect(joining(delimiter: "\n\n"));  
}
```

//检查某模块是否已经装载

```
public static boolean isModulePresent(String moduleName) {  
    return ModuleLayer  
        .boot()  
        .findModule(moduleName)  
        .isPresent();  
}
```

```
private static void queryLayer() {  
    var bootLayerInfo :String = describe(ModuleLayer.boot());  
    System.out.println(bootLayerInfo);  
    //查询指定的模块是否已经加载  
    System.out.println(isModulePresent(moduleName: "jdk.crypto.ec"););  
}
```

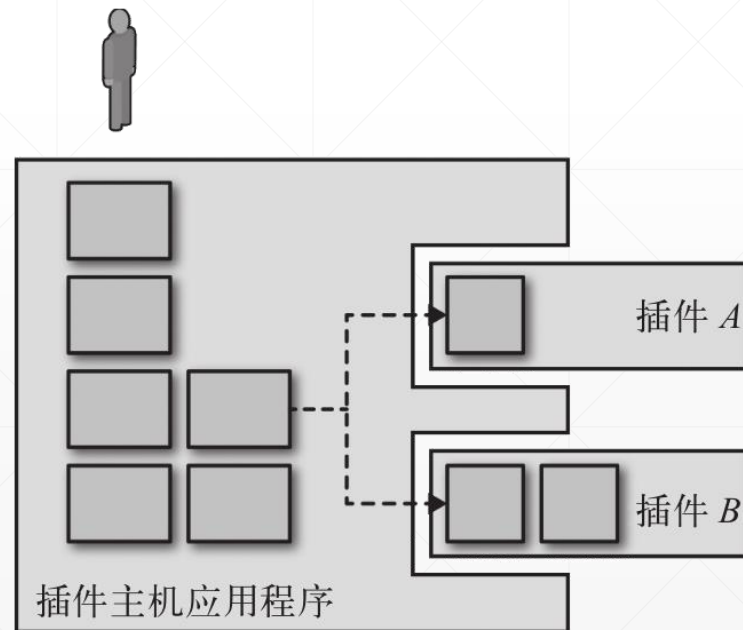


```
Run: Main x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
module java.naming  
module jdk.crypto.mscapi  
module jdk.crypto.cryptoki  
module jdk.compiler  
module jdk.crypto.ec  
module jdk.security.jgss  
module jdk.internal.opt  
module java.compiler
```

基于层实现“插件化”系统

通常情况下，普通的Java应用程序是用不着层的，但如果你正在开发一个框架，这个框架支持动态地加载插件，这时，就可以通过扫描特定的插件文件夹，创建新层动态地装载这一文件夹中发现的插件（其实就模块），从而实现这种易于扩展的“插件架构”。

《Java 9模块化开发》第6章给出了一个例子，可供参考。



下一讲



模块的传递性与聚合模块